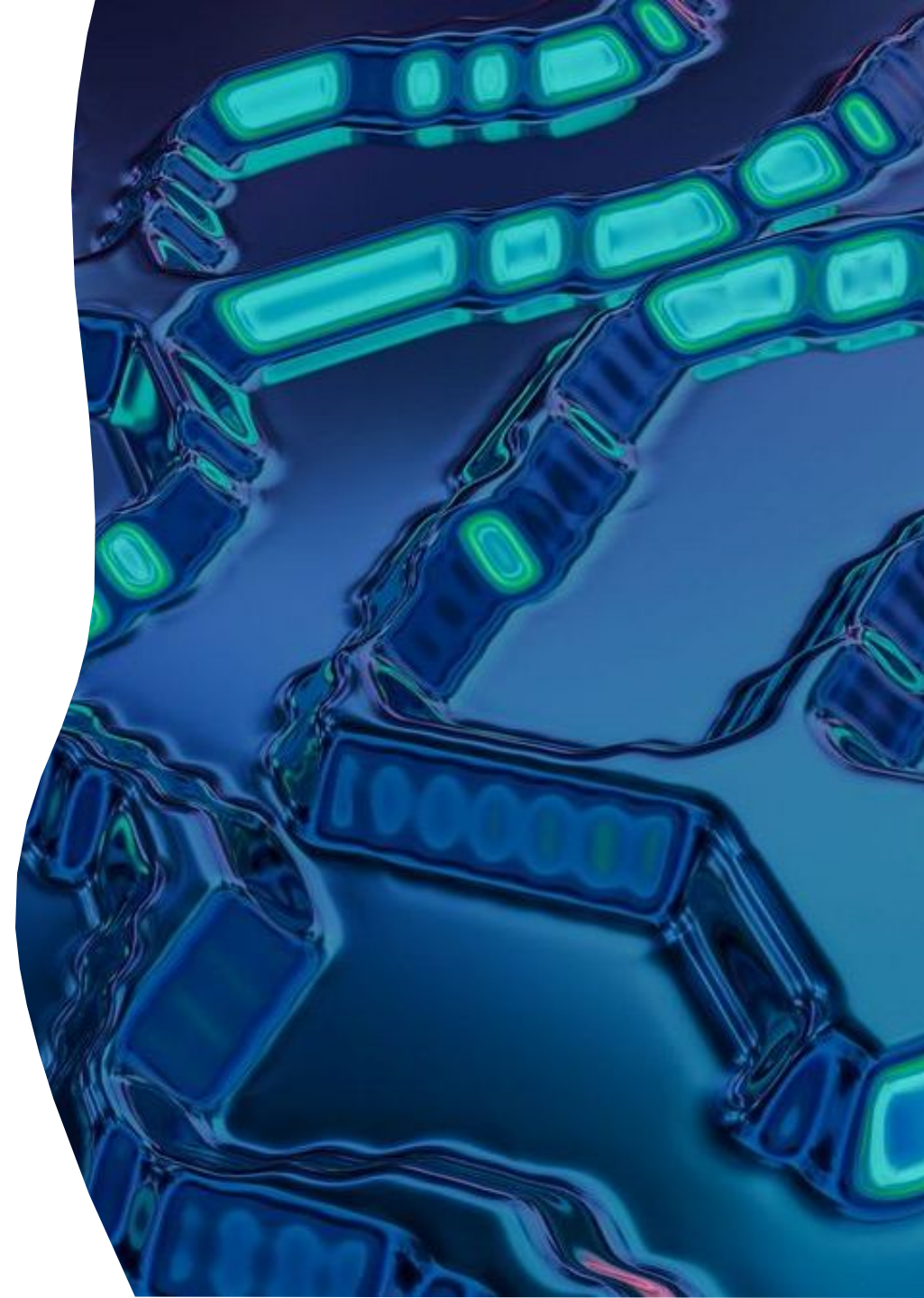


Data Analytics, Data Mining and Machine Learning

Lecture 4

Narges Chinichian · Semester 1, 2026



Overview of this course

Week	Date	Topic
1	07.09.2026	Introduction
2	14.09.2026	Data, Research Questions & Ethics
3	21.09.2026	Data Visualisation & Exploratory Data Analysis
4	28.09.2026	Preprocessing & Feature Engineering
5	05.10.2026	Machine Learning Fundamentals & Regression
6	12.10.2026	Project Presentations I
7	19.10.2026	Classification: Logistic Regression, KNN & Decision Trees
8	26.10.2026	Bank Holiday — No Class
9	02.11.2026	Gradient Descent, Regularisation & Model Complexity
10	09.11.2026	Model Evaluation & Ensemble Methods
11	16.11.2026	Neural Networks, Unsupervised Learning & Special Topics
12	23.11.2026	Project Presentations II

Quick Quiz

Check Loop



Today's Challenge

We have a dataset. Can we give it directly to all sorts of algorithms?

 What is wrong with this data?

age	income	education	gender
23	32000	Bachelor	Female
NaN	45000	bachelor	female
999	-5000	Master	F
31	NaN	PhD	Male

From Data to a Model

Training Data

Used to **learn** the model: the algorithm sees these observations to find patterns.

Test Data

Used to evaluate how well the model performs on **unseen data**.
Look at it once, at the end.



Why Do We Need a Test Set?

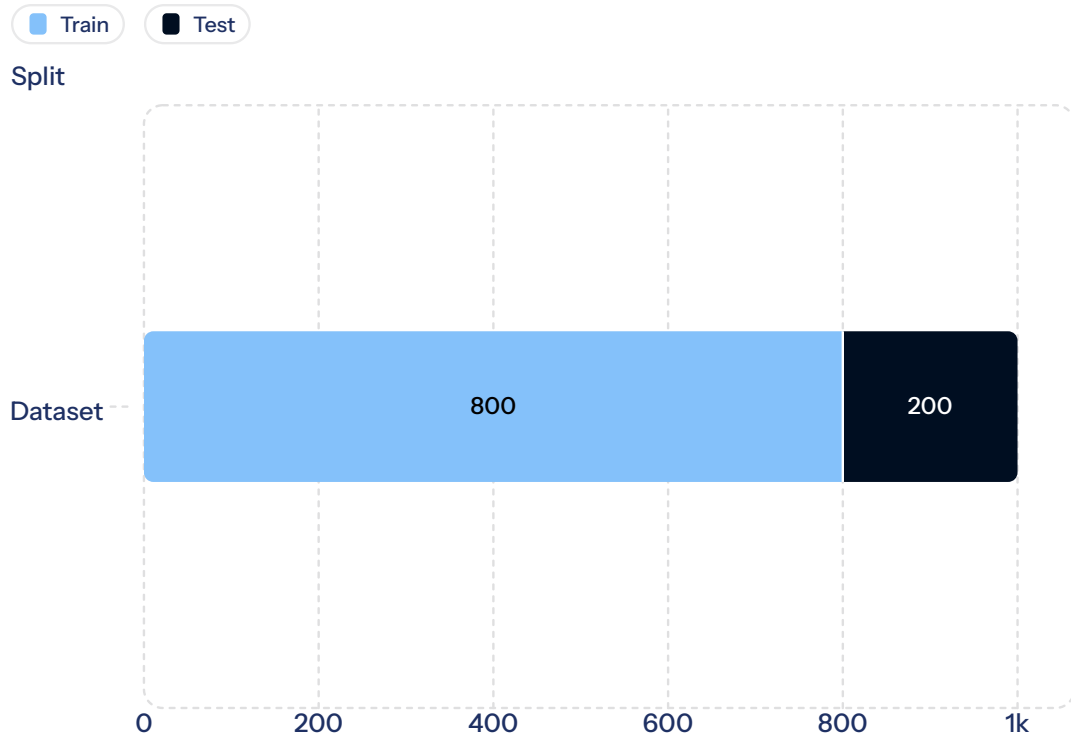
Imagine studying for an exam using exactly the questions that will appear. Getting 100% tells us nothing about whether you actually learned to use the material independently.

Machine learning has the same problem. We want to know:

**Can the model generalise to data it
has never seen before?**

The test set gives us an approximation of this.

A Simple Example: 80/20 Split



How it works

With **1,000 observations**, a common starting point is an **80/20 split**.

- 800 observations used to **train** the model
- 200 observations remain **unseen** until evaluation

The appropriate split depends on the problem and dataset size.



The Fundamental Rule

Never train on the test set.

If the model has access to test answers during training, the test set is no longer an honest measure of generalisation. But there is another, less obvious problem...

Why learn this now?

Preprocessing Can Also Leak Information

Suppose we want to standardise income. We need a **mean** and **standard deviation**. Where should these come from?

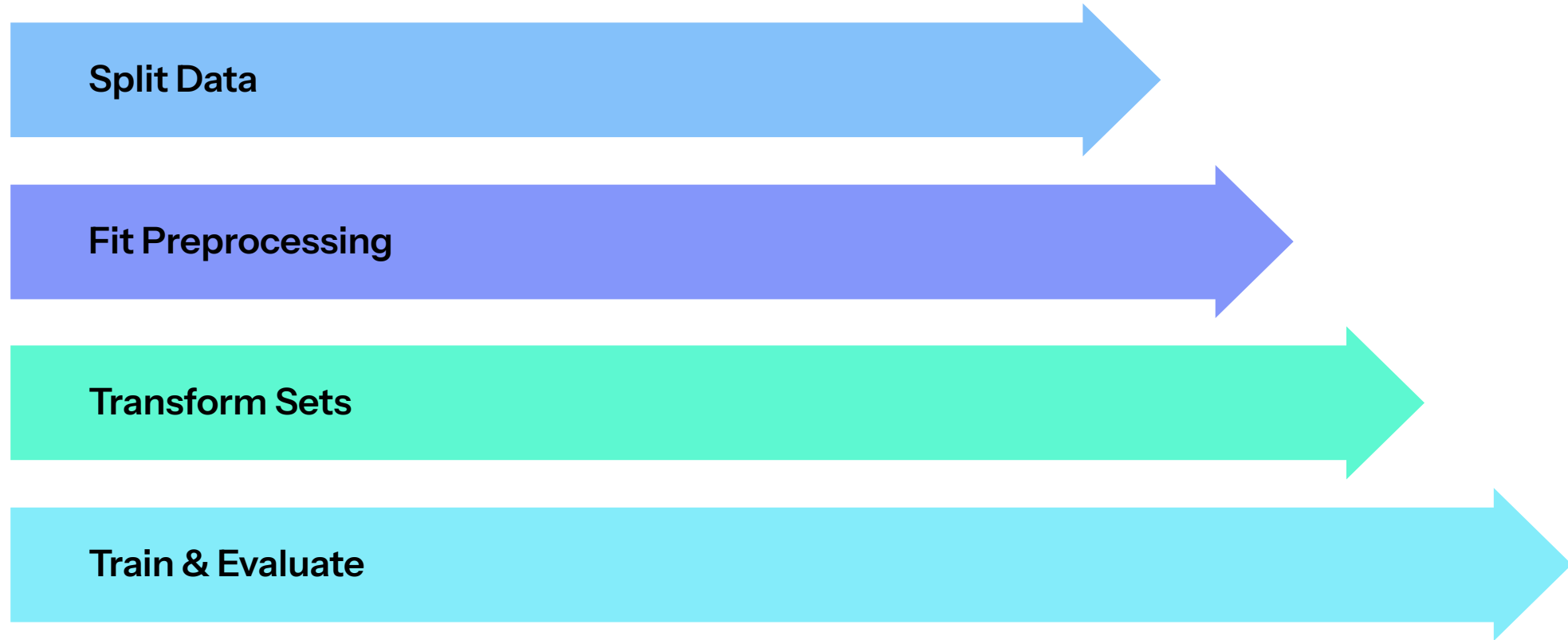
Option A 

Calculate using **all data**, including the test set. This leaks information from the test set into preprocessing.

Option B 

Calculate using **training data only**. This preserves the idea of a truly unseen test set.

The Correct Workflow



❏ **Fit** preprocessing on training data. **Apply** the fitted preprocessing to test data.

What Does "Fit" Mean?

StandardScaler

Learns **mean** and **standard deviation** from training data

Median Imputation

Learns the **median** value from training data

One-Hot Encoding

Learns **which categories exist** in training data

Transform = use those learned parameters to change the data. Fitting and transforming are two distinct steps.

Example: Scaling Income

Training data

Income values: €30k, €40k, €50k, €60k

We calculate **mean and standard deviation from training data only**, then use those same values to transform both sets.

Key rule

TRAIN

Fit + Transform using training statistics

TEST

Transform only: using the same training statistics

Why Is This Called Data Leakage?

Data leakage = information from outside the training data influences the model-building process.



Where leakage hides

Preprocessing, feature engineering, variable selection, imputation, scaling, feature construction



The general principle

Information from the test set must **not** influence decisions made during training

Target Leakage

Target leakage occurs when a feature contains information about the target that wouldn't be available at prediction time.

What it looks like

Loan Default

account_closed (known only after default)

Disease Prediction

prescribed_medication (prescribed because of the disease)

Customer Churn

cancellation_date (exists only after churn)

Why it's dangerous

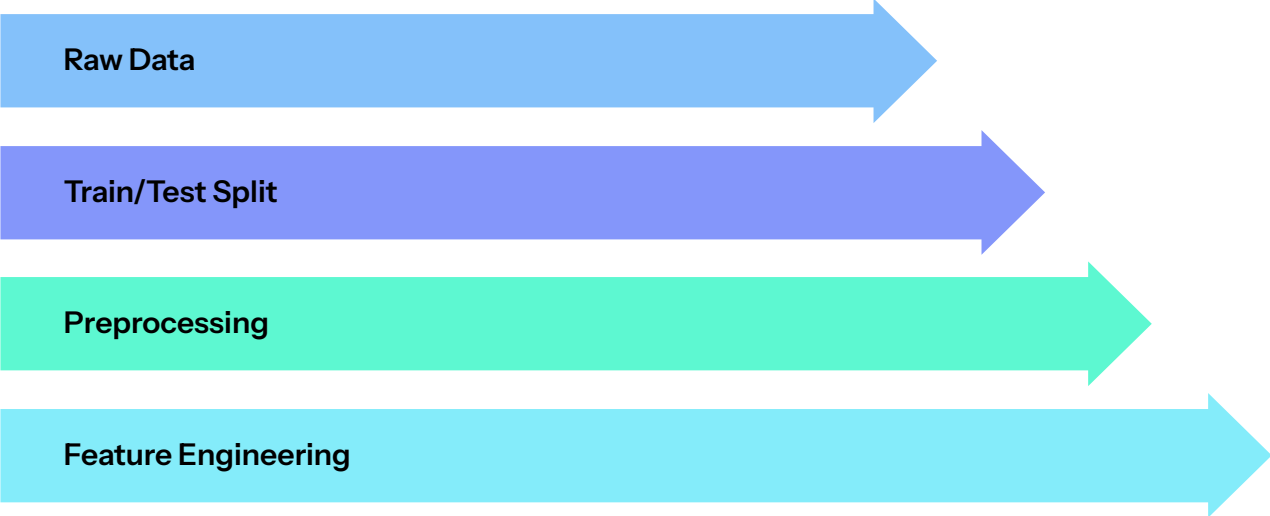
- The model learns a spurious shortcut — train and test scores both look excellent, because the leak is present in both splits.
- The problem only surfaces in deployment, when the future data doesn't contain the leaked feature.



Always ask: Would this feature be available before the outcome is known? If not, remove it.

Now: Preprocessing

With that principle in mind, we can start preparing our data. But what problems might we have to solve first?



Raw Data

Train/Test Split

Preprocessing

Feature Engineering



Why preprocessing?



Missing Values



Impossible Values



Duplicate Records



Inconsistent Categories



Different Scales



Outliers

Also: dates, text, and variables not represented in a useful way. **Investigate before changing anything.**

Cleaning vs Feature Engineering

Data Cleaning

Finding and handling problems in the data.

- Missing values
- Duplicate records
- "Female", "female", "F" → standardise
- Age = 999 → investigate

Feature Engineering

Creating useful representations from existing information.

- date → year / month / day_of_week
- income ÷ household_size → income_per_person

Missing Values

Missing $\neq$ zero $\neq$ unknown $\neq$ not applicable. The right strategy depends on the data and research question.

- 1 Remove rows
- 2 Remove the variable
- 3 Impute with mean or median
- 4 Most frequent category
- 5 Create an "Unknown" category

📌 There is no universally correct choice.

Outliers



Unusual $\neq$ Incorrect

Age = 237 ❌

Probably an error — investigate and correct

Income = €250,000 ✅

Unusual but plausible: do not remove without cause

Before removing an outlier: investigate it.

Categorical Variables

Many ML algorithms require numerical input, so categorical variables must be **encoded**. But before choosing an encoding method, ask: **is this variable nominal or ordinal?** The distinction matters.

Nominal

Categories with **no natural order**: no meaningful ranking between values.

- Country
- Gender
- Colour

Ireland, Germany, Palestine, none is "greater than" another.

Ordinal

Categories with a **natural order**: values can be meaningfully ranked.

- Education level
- Customer satisfaction
- Disease severity

Bachelor < Master < PhD: the order carries real meaning.

Categorical Variables 1: Nominal Encoding

For **nominal** variables, categories have no natural order. We use **One-Hot Encoding** to represent them numerically without implying any ranking.

One-Hot Encoding

Converts each category into a **binary column**: a 1 indicates the category is present, and 0 means it is not. Suitable for **nominal** variables where there is no natural order. There is no meaningful ranking between country names: no Ireland < Germany < Palestine.

ID\Country	Ireland	Germany	Palestine
ID1	1	0	0
ID2	0	1	0
ID3	0	0	1

Categorical Variables 2: Ordinal Encoding

For **ordinal** variables, categories have a natural order. We use **Ordinal Encoding** to assign integers that preserve the ranking.

Ordinal Encoding

Assigns an integer to each category in a meaningful order. The numbers reflect the rank, higher means "more" of the quality being measured. Suitable for **ordinal** variables where the order carries real meaning.

Example: Education level: High School < Bachelor's < Master's < PhD

Education Level	Encoded Value
High School	1
Bachelor's	2
Master's	3
PhD	4

 Unlike one-hot encoding, ordinal encoding preserves the ranking between categories. However, it implies equal spacing between levels, use with care when that assumption doesn't hold.

Scaling Numerical Variables

The problem

Variables live on very different scales:

- Age: 18–80
- Household size: 1–6
- Income: 20,000–150,000

Standardisation

$$z = \frac{x - \mu}{\sigma}$$

Distance-based methods (k-NN, k-means) need scaling; otherwise, income dominates age.

Tree-based models (decision trees, random forests) don't; they split one feature at a time.

Feature Engineering

Feature engineering = creating useful features from existing information. Good feature engineering often requires domain knowledge.

income ÷ household_size

→ income_per_person

survey_date

→ year / month / day_of_week

text field

→ word_count / text_length

Ask: **What information is hidden inside the variables we already have?**

Our Workflow Today

01	02
Inspect	Remove Duplicates
03	04
Train / Test Split	Clean*
05	06
Handle Missing Values	Investigate Outliers
07	08
Process Dates	Engineer New Features
09	10
Encode Categories	Scale Numerical Variables
11	
Build a Preprocessing Pipeline	

*Anything that learns from the data goes after the split, and pure row-wise corrections can come before.

The Way We Will Work

Observe

What do we see?

Think

What might be causing it?

Try

What could we do?

Discuss

What are the trade-offs?

Code

Implement the solution.

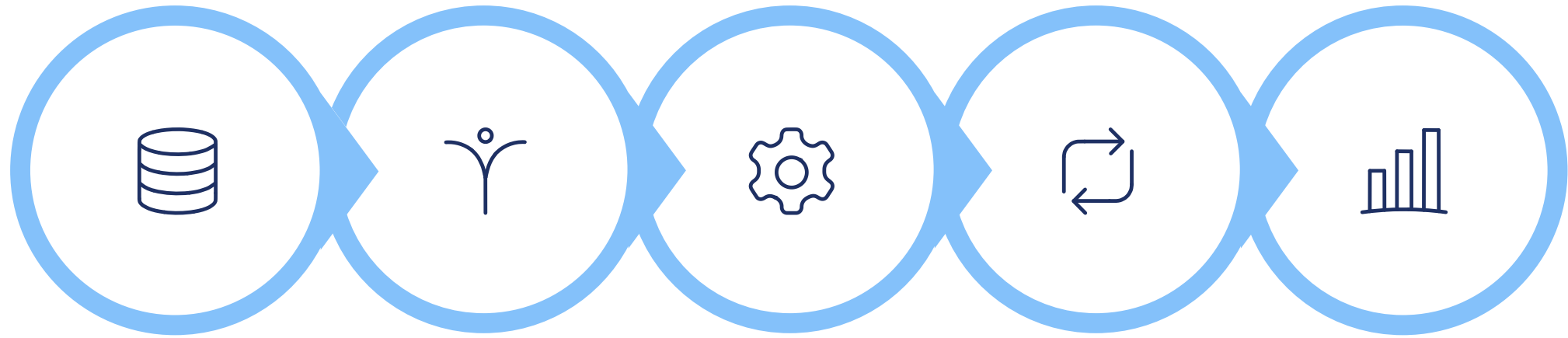
Let's Work With the Data

Open the notebook.

Our goal is not to make the dataset "perfect". Our goal is to make **defensible preprocessing decisions**.



The Final Workflow



Raw Data

Train / Test Split

Fit
Preprocessing

Transform Train
& Test

Train & Evaluate

Fit preprocessing

On training data only

Transform

Both train and test using fitted parameters

Evaluate

Model performance on the unseen test set